

HIRIX: AI RESUME SCREENING SYSTEM

PROJECT SYNOPSIS

OF Training 104

BACHELOR OF TECHNOLOGY

Computer Science and Engineering

(2022-2026)

SUBMITTED BY

AMANDEEP KAUR

CRN: 2215008

URN: 2203395

(June 2026)



GURU NANAK DEV ENGINEERING COLLEGE,

LUDHIANA

TABLE OF CONTENTS

Sr. No.	Content	Page No.
1.	Introduction	1
2.	Rationale	2
3.	Objectives	3
4.	Literature Review	4
5.	Feasibility Study	5
6.	Methodology / Planning Of Work	6
7.	Facilities Required For Proposed Work	8
8.	Expected Outcome	9
9.	References	10

1. INTRODUCTION

Hirix AI Resume Screening and Scoring System is a state-of-the-art recruitment intelligence tool that automates the matching of candidate resumes with job descriptions. With hundreds of applicants for each vacancy in one big human resource department, manual screening bottlenecks and recruiter fatigue are common against a backdrop of change in the company. With Hirix, information extraction features calculation and relevancy scores from candidates are housed within an automated, intelligent pipeline. The system has a web interface that recruiters use to upload several resumes as batches, view the list of applicants ranked from most relevant to least relevant, participate in changing screening settings depending on different publications, and later export automated results for further rounds of interviews.

The proposed system features a unique two-stage pipeline that aims to minimize computational cost while retaining high screening accuracy. In first stage is a quick keyword + word-count overlap on the unprocessed text that we pull from PDF resumes. Resumes that don't meet a bare minimum relevance threshold are immediately filtered out so as not to waste computing power on close-reading them. Any resume that makes it through this phase is then run through an additional second-phase system that internally compares recently parsed resumes for named entity recognition, experience duration parsing, degree mapping and semantic vector similarity. This dual-layered strategy ensures that the system can process large volumes of applications quickly and efficiently. An ensemble of machine learning models (Logistic Regression, Random Forest, XGBoost) is optionally used for more accurate relevance scoring.

2. RATIONALE

Traditional hiring workflows are extremely unproductive and inefficient because of the number of applications set in motion on online portals. From our day-to-day experience with HR teams, we see that the standard practice is to spend dozens of hours on end reviewing a resume for just one position which is tedious and traditional; but also prone to human error, cognitive fatigue and unconscious bias or bias super-exploitation. This bottle-neck lengthens the time-to-hire and consequently results in lost productivity alongside a competitive talent pipeline moving at a faster pace, leading to potentially losing high-quality talent. In addition, simple keyword-matching applicant tracking systems (ATS) tend to be too strict: they eliminate qualified candidates who describe their experience with synonyms or different wording from those used in the job description. This kind of screening mechanism that is objective, contextual and fast is what most organizations need today in an intelligent automated manner.

The first pass-course filter improves operational efficiency by instantly discarding resumes from totally irrelevant, whereas the fine parsing stage determines applications that passed the initial course filters across multiple structured heads to guarantee a best fit. Implementing this system decreases the cost of operations in screening, takes human bias out from the first screen stage, speeds up your recruitment pipeline and gives recruiters back their time to hit candidate. Deploying this system significantly reduces the operational cost of streaming as it tackles all areas in which they are primarily challenged.

3. OBJECTIVES

1. To develop an AI-powered resume screening system capable of extracting textual data from PDF resumes.
2. To automate the extraction of candidate attributes and perform intelligent ranking and classification based on job requirements.
3. To implement a two-stage resume screening architecture that combines coarse filtering and detailed evaluation for efficient candidate assessment.

4. LITERATURE REVIEW

Automated resume screening, which originated in applicant tracking systems (ATSs), began with simple keyword matching algorithms that parsed resumes for exact matches on character strings. To overcome this, modern software engineering has hybrid text-extraction engines that use fast vector parsing via PyMuPDF, accurate layout reconstruction using pdfplumber as well as Optical Character Recognition fallback libraries like Tesseract for high-fidelity text, irrespective of encoding.

- Unaided keyword matching would be the cornerstone of early recruitment tracking systems leaving screening accuracy equally poor as they often missed synonyms, descriptions of conceptual skills or intent behind skill.
- Data extraction was prone to data loss from various formats of resumes, i.e tables, columns and scanned images This is accounted for by modern hybrid parsers which utilizing PyMuPDF, pdfplumber and OCR wrappers (Tesseract) in a single approach.
- Recruiters are able to use natural language processing (NLP) tools, including named entity recognition (NER) frameworks such as spaCy, for more than just analyzing flat text; they can now parse out specific entities (e.g., skill tags, experience duration, academic degree) from flat text files.
- By using pre-trained models, recruiters are able to map paragraphs to a high dimensional space, allowing for the recognition of phrases that have different wording, but share the same professional intent.

As Natural Language Processing (NLP) and Computational Linguistics developed, researchers also began using Named Entity Recognition (NER) to extract structured data about unstructured resume text.

5. FEASIBILITY STUDY

The feasibility study of the proposed AI Resume System shows that the project is practical, useful, and beneficial for companies.

- **Feasibility:** A feasibility study is the first step in the Software Engineering Process, examining if the idea for a system is viable and will provide value. This feasibility study looks at the core business problems, identifies the technical requirements.
- **Need of the Project:** This project aims to address the current disruption in today's recruitment process due to bottlenecks (e.g., a backlog of job applications) that HR departments cannot keep up with due to having an unprecedented number of applications come in daily. Screening resumes manually is a very slow, labor-intensive, and error-prone process that causes candidate fatigue and has increased time-to-hire, creating a poor candidate experience and reducing an employer's ability to attract top talent. Meanwhile, current Applicant Tracking Systems (ATS), which are used by employers as part of their recruiting processes are not adequately addressing the needs of hiring managers because they rely on exact matches of keywords; therefore, they commonly disqualify candidates who legitimately meet the requirements for employment but have used synonyms or alternate language to describe themselves or their former responsibilities.
- **Significance of the Project:** Project significance is due to the ability to convert Talent Acquisition from a cumbersome, subjective process into a lean, data-driven, and objective workflow. The use of a 2-phase screening pipeline will enable companies to inspect thousands of applications in minutes and continue to meet the high standards of evaluation accuracy. The methodology for selecting candidates based on conceptual capabilities through semantic matching and machine learning models.

6. METHODOLOGY / PLANNING OF WORK

The way the Hirix AI Resume Screening and Scoring System's work is managed is through a stage-by-stage software engineering approach that consists of three main parts, namely design, development, and testing. The project is split into a number of sequential phases, starting with requirement gathering and architectural planning; moving onto modular development of the extraction, features, and matching engines; and finally, integration of the user interface and system testing. The methodology used in this approach will be one of incremental development, allowing for flexibility in terms of iteration and how the application will be developed. The project is to proceed in phases as outlined below:

Phase 1 – Requirement Analysis (One-Time)

- All system requirements were identified upfront — text extraction, preprocessing, feature extraction, JD processing, matching, ranking, ML scoring, and UI.
- Requirements were divided into increments based on module dependency and priority.

Phase 2 – Increment Planning

- Each increment was planned to deliver one or more independent, functional modules.
- Dependencies between modules determined the increment order (e.g., text extraction before feature extraction).

Phase 3 – Design, Code, Test (Per Increment)

- Each increment followed its own mini design → development → testing cycle.
- Modules were tested independently using dedicated test scripts before integration.

Phase 4 – Integration and Delivery (Per Increment)

- After testing, each new module was integrated into the existing system.
- A working version of the system was available after every increment.

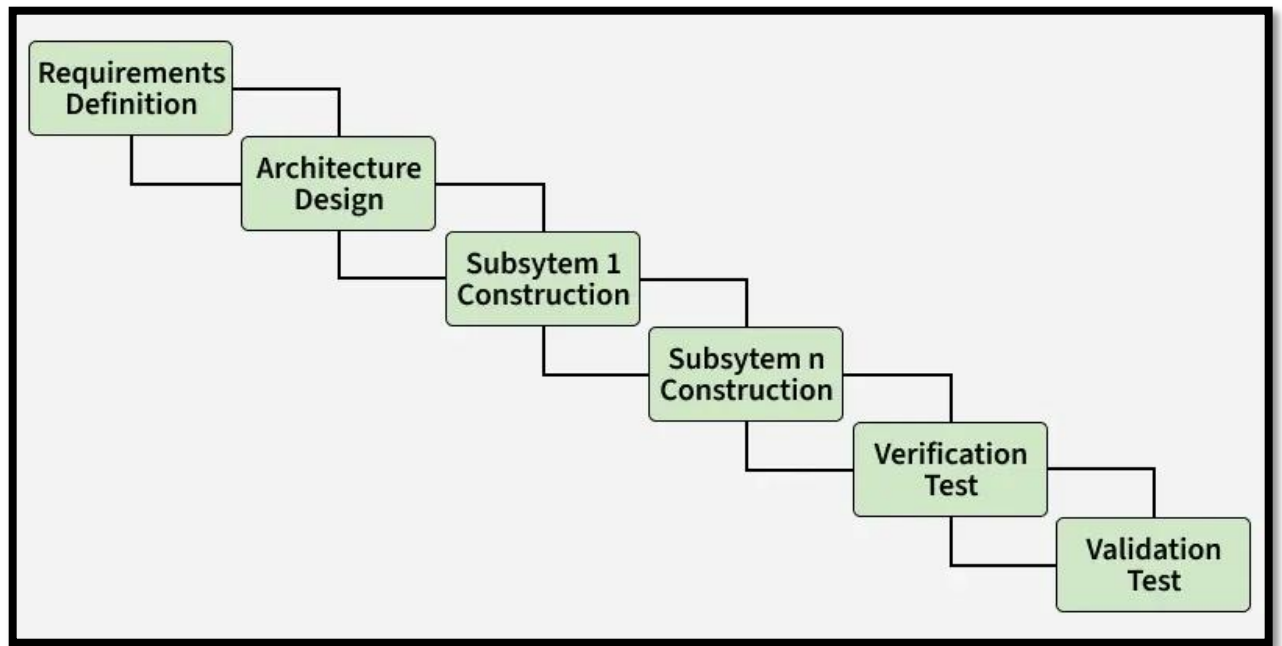


Figure 1: Methodology/ Planning of work

7. FACILITIES REQUIRED FOR PROPOSED WORK

Hardware Requirements:

- A current-generation multi-core CPU (e.g. Intel Core i5/i7; AMD Ryzen 5/7; Apple M-series).
- At least 8 GB of RAM (ideally 16 GB), which is required to support the computationally intense requirements of NLP model vocabulary size, Transformer sentence embeddings, and batch processing high volumes of resumes without experiencing a crash or other issues related to memory limits.
- At least 10 GB of empty SSD space will be prior to installing a Python virtual environment, caching any necessary pre-trained machine learning models, and storing the entire collection of resumes.

Software Requirements:

- This project requires the use of a common OS (Windows, macOS or Linux) running Python 3.8 - 3.11 for development.
- Text will be extracted from both digital and scanned resumes using PyMuPDF and pdfplumber, while Tesseract will serve as an external OCR backup engine for any image based PDFs.
- Analytically, the processing and matching logic are powered by a stack of NLP and Machine Learning tools, include spaCy, NLTK, Scikit-learn, sentence-transformers, and XGBoost.
- The interactive dashboard was developed with Streamlit, using Plotly and Matplotlib for visual reporting, and with pandas and openpyxl used for data processing.

8. EXPECTED OUTCOME

The project will result in a Streamlit dashboard which would allow an employer or recruiter to enter job parameters (Type of job), upload multiple PDF resumes, and receive an instant ranked list of applicants. The candidate list will show exact skill badges that match and don't match with the job to be filled along with experience analysis and academic degree verification. This applicant categorization will be done using set thresholds, as no 2 recruiters will use the same thresholds for categorizing applicants as either "relevant" or "non-relevant", and there will also be detailed log entries on what made the candidate "not-relevant" for that particular employer/recruiter. The visual metrics for candidates will be through Plotly charts; for example, score spreads and skills coverage, distribution of experience scores, and there will be an option to download candidates ranked in an Excel spreadsheet.

To achieve these goals, the system provides a fast and efficient way to process candidate profiles, focusing on reducing overall time spent recruiting. The first stage of processing initiates by performing a gross or coarse search on the candidate's profile, which allows the candidate to be filtered out quickly based on job criteria and processed with minimal latency (milliseconds). This will allow companies to reduce their time to hire significantly for high-volume candidates. The second stage completes processing by providing precise ranking of candidates based on their level of similarity to the job specification via rule-based matching, NLP contextual similarity scores, and machine-learning models (ensemble prediction). Benefits include reducing the recruiter's workload substantially, removing manual sorting bias from candidate selection, and increasing the likelihood of finding suitable job candidates who meet all of the desired criteria by enhancing the accuracy of top-performing candidates.

References

- [1] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. Sebastopol, CA, USA: O'Reilly Media, 2009.
- [2] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.
- [3] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.
- [4] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [5] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERTnetworks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Hong Kong, China, 2019, pp. 3982–3992.
- [6] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [7] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [8] A. Vaswani et al., “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.
- [9] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT*, Minneapolis, MN, USA, 2019, pp. 4171–4186.